OPIPE-05: Business & Security Event Pipelines

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 5 of 7 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Processing Business Transactions and Security Events at Ingestion

Business events and security events are two OpenPipeline scopes with distinct governance requirements. Business events drive revenue analytics, conversion tracking, and SLO calculations. Security events drive compliance reporting, threat detection, and audit trails. Both benefit from ingestion-time processing — enrichment, routing, and metric extraction — but the rules are different.

This notebook covers the Business Events and Security Events scopes in OpenPipeline. For log processing, see **OPLOGS-01: OpenPipeline Fundamentals**. For compliance and masking patterns, see **OPLOGS-08: Security & Data Protection** and **OPMIG-08: Security, Masking & Compliance**.

Table of Contents

1. Business Events Scope
2. Business Event Enrichment
3. Business Event Metric Extraction
4. Security Events Scope
5. Security Event Routing and Compliance
6. Event-to-Metric Extraction for KPIs
7. Summary
8. Next Steps
9. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and business events enabled

Requirement	Details
Permissions	`storage:bizevents:read`, `storage:security.events:read`; `settings:objects:write` on the `builtin:openpipeline.bizevents.*` / `builtin:openpipeline.security.events.*` schemas to configure pipelines (OpenPipeline configuration is stored as Settings objects)
Data	Business events ingested via OneAgent capture rules, the RUM API, the business events API, or OpenPipeline extraction
Recommended	**OPIPE-01** (multi-scope architecture), **BIZEV** series for business event analytics

1. Business Events Scope

Business events represent **user transactions and business actions** — purchases, sign-ups, page views, API calls, form submissions. They are ingested through:

- **OneAgent** — capture rules on incoming requests to monitored services
- **RUM (web and mobile)** — an explicit call to the RUM JavaScript API, OneAgent for Mobile, or OpenKit; user actions are not turned into business events automatically
- **Business events API** — JSON events sent from external systems and backend services
- **OpenPipeline** — business events extracted from logs and spans (Data extraction stage)
- **Workflows** — the *Ingest business event* action

Key Business Event Fields

Field	Description	Example
`event.type`	The business event type	`com.myapp.purchase.completed`
`event.provider`	The source system	`www.myshop.com`
`event.category`	Event classification	`purchase`, `signup`, `pageview`
`event.group`	Logical grouping	`checkout-flow`, `onboarding`
Custom attributes	Business-specific data	`order.total`, `product.category`, `user.tier`

Discovering Your Business Event Landscape

```
// Business event volume by type and provider
fetch bizevents, from:-24h
| summarize event_count = count(), by:{event.type, event.provider}
| sort event_count desc
| limit 20
```

```
// Business event trend by type over 24h
fetch bizevents, from:-24h
| makeTimeseries event_count = count(), by:{event.type}, interval:1h
```

2. Business Event Enrichment

Business events often arrive with technical identifiers that need business context. OpenPipeline enrichment adds meaningful fields at ingestion.

Common Enrichment Patterns

Condition	Enrichment Field	Value	Purpose
`matchesValue(event.type, "*purchase*")`	`business.domain`	`"commerce"`	Domain classification
`event.provider == "mobile-app"`	`channel`	`"mobile"`	Channel attribution
`event.provider == "www.myshop.com"`	`channel`	`"web"`	Channel attribution
Custom: `order.total > 1000`	`order.tier`	`"high_value"`	Revenue tier classification
Custom: `user.country == "US" or user.country == "CA"`	`region`	`"north_america"`	Geographic grouping

Security Context for Business Events

Sensitive business data (revenue figures, user details) should be access-controlled:

Condition	`dt.security_context` Value	Rationale
`matchesValue(event.type, "*payment*")`	`"finance-team"`	Payment data restricted to finance
`matchesValue(event.type, "*signup*")`	`"marketing-team"`	User acquisition data for marketing
`true` (last)	`"product-team"`	General product analytics

The Permission stage runs **first match only**, so list the specific conditions first and the `true` catch-all last.

3. Business Event Metric Extraction

Extract metrics from business events for dashboards, SLOs, and alerting.

Example: E-Commerce Metrics

Metric Key	Processor	Condition	Dimensions
`bizevent.purchase_count`	Counter metric	`event.type == "purchase.completed"`	`event.provider`, `channel`

Metric Key	Processor	Condition	Dimensions
`bizevent.purchase_revenue`	Value metric on `order.total`	`event.type == "purchase.completed"`	`event.provider`, `channel`
`bizevent.cart_abandonment`	Counter metric	`event.type == "cart.abandoned"`	`event.provider`
`bizevent.signup_count`	Counter metric	`event.type == "user.signup"`	`channel`, `region`

Conversion Funnel Metrics

Track progression through a multi-step funnel by extracting a count metric at each stage:

Funnel Stage	Event Type	Metric Key
Product viewed	`product.viewed`	`funnel.stage_1_views`
Added to cart	`cart.item_added`	`funnel.stage_2_adds`
Checkout started	`checkout.started`	`funnel.stage_3_checkout`
Purchase completed	`purchase.completed`	`funnel.stage_4_purchase`

```
// Conversion funnel from business events (last 24h)
fetch bizevents, from:-24h
| summarize views = countIf(event.type == "product.viewed"),
    adds = countIf(event.type == "cart.item_added"),
    checkouts = countIf(event.type == "checkout.started"),
    purchases = countIf(event.type == "purchase.completed")
```

4. Security Events Scope

Security events include threat detections, vulnerability findings, compliance findings, and events from external security tools. They are stored in their own table, **`security.events`** — not in `events` — so query them with `fetch security.events`. They require special handling:

- **Never drop** — Security events must be retained for compliance, even if volume is high
- **Never sample** — Every security event is potentially significant
- **Long retention** — compliance frameworks and internal policy commonly call for a year or more (see §5)
- **Strict access control** — Only security teams should access security event data

Key Security Event Types

Category	`event.type`	Description
Vulnerabilities	`VULNERABILITY_FINDING`, `VULNERABILITY_SCAN`, `VULNERABILITY_STATE_REPORT_EVENT`, `VULNERABILITY_STATUS_CHANGE_EVENT`, …	Vulnerability findings and state from Dynatrace and third-party tools. `VULNERABILITY_STATE_REPORT_EVENT` and `VULNERABILITY_STATUS_CHANGE_EVENT` are two of the four types that stay non-configurable in OpenPipeline from SaaS 1.353 (§5)
Detections	`DETECTION_FINDING`	Alerts from security tools; Runtime Application Protection attacks carry `product.name == "Runtime Application Protection"`
Compliance	`COMPLIANCE_FINDING`, `COMPLIANCE_SCAN_COMPLETED`	Security Posture Management scan results

> **Sources:** Threat Observability concepts (DT docs) — event-type tables per category.
>
> **Platform audit events are not security events.** Configuration changes and user actions are recorded as `AUDIT_EVENT` records in `dt.system.events`, outside the security-events scope.

```
// Security event overview (security events are stored in security.events, not events)
fetch security.events, from:-24h
| summarize event_count = count(), by:{event.type}
| sort event_count desc
```

5. Security Event Routing and Compliance

Security events should always be routed to **dedicated buckets** with extended retention and restricted access.

Routing Strategy

Pipeline	Routing Condition	Target Bucket	Retention	Security Context
`vulnerability-events`	`matchesValue(event.type, "VULNERABILITY_*")`	`security_vulnerabilities`	365 days	`"security-team"`
`detection-findings`	`event.type == "DETECTION_FINDING"`	`security_detections`	365 days	`"security-team"`
`compliance-events`	`matchesValue(event.type, "COMPLIANCE_*")`	`compliance_events`	2555 days (7 years)	`"compliance-team"`

Platform audit events (`AUDIT_EVENT`) are not in this scope — they live in `dt.system.events` (§4). Which security event types pass through your pipelines depends on how they are ingested, so check the routing

preview against real records before relying on a route.

Today vs from SaaS 1.353

Your pipelines can route only part of `security.events` today. The security-events migration guide stores events Dynatrace generates in `default_securityevents_builtin` (*"Three years, fixed"*) and events ingested from third-party tools in `default_securityevents` (*"One year, configurable"*). It lists routing to custom buckets among the changes that *"come with the new bucket"*. So until SaaS 1.353 reaches your tenant, the routes above shape third-party events only. Built-in events, such as the `COMPLIANCE_*` events from Security Posture Management, stay in their default bucket.

Security events	Until SaaS 1.353 reaches your tenant	From SaaS 1.353
Generated by Dynatrace	`default_securityevents_builtin`, three years, fixed; not routed by your pipelines	`default_security`, 35 days, configurable; configurable and routable in OpenPipeline, except the four vulnerability types in the note below
Ingested from third-party tools	`default_securityevents`, one year, configurable; the routes above apply	`default_security`, 35 days, configurable; the routes above apply

Tell the two kinds apart by `dt.openpipeline.source`, not by bucket name: *"Every event Dynatrace generates has the dt.openpipeline.source field set to built-in-security-events, which separates the two kinds of security event wherever each one is stored."* To route only third-party events, the guide's routing condition is `dt.openpipeline.source != "built-in-security-events"`; for built-in events, use `==`. Add it to the conditions in the table above, and *"Place your route before the built-in security.events pipelines so that it runs first."* The field is already set, so queries can switch from bucket filters to it now. The query after this section shows which kind you hold and where it is stored.

> **Sources:** Migrate to the new security.events bucket setup (DT docs) — *"Filter on dt.openpipeline.source instead."* **Derived:** that built-in events cannot be routed before 1.353 follows from the guide listing routing to custom buckets among the changes that come with the new bucket.
>
> **Forthcoming — SaaS 1.353 (planned December 2026), announced in the SaaS 1.350 release notes:** *"With Dynatrace SaaS version 1.353, planned for December 2026, security events become fully configurable in OpenPipeline, and security event storage consolidates into a single Grail bucket with a new default retention period." "All events will flow into the new default_security bucket automatically."* Four vulnerability types are excepted: *"VULNERABILITY_STATE_REPORT_EVENT, VULNERABILITY_ASSESSMENT_CHANGE_EVENT, VULNERABILITY_TRACKING_LINK_CHANGE_EVENT, and VULNERABILITY_STATUS_CHANGE_EVENT remain non-configurable, because a future release will deprecate them in favor of native concepts."* A `VULNERABILITY_*` route such as `vulnerability-events` above therefore does not cover those four once 1.353 lands; their retention is set on `default_security`. Act before it reaches your tenant if you query security events by bucket name — *"Notebooks, dashboards, and workflows filtering on default_securityevents or default_securityevents_builtin need updating."* — or if *"You rely on retention beyond 35 days."* — *"Either extend the retention of default_security, or add an OpenPipeline routing rule sending the data you need to keep to a longer-retention bucket."* Until SaaS 1.353 reaches your tenant, the routing above remains the working path.

> **Sources:** What's new in Dynatrace SaaS 1.350 (DT docs) — *"Rollout start on Nov 4, 2026"*. **Derived:** the four excepted types cannot be routed to a custom bucket, so `default_security` retention is the only lever the note leaves for them.

Compliance Framework Requirements

The periods below are the ones commonly cited for each framework, not legal advice — confirm the period that applies to you with your compliance team and auditor before you size a bucket:

Framework	Commonly cited retention	Key Event Types
SOC 2	No fixed period in the framework — set by your policy and auditor; one year is common	Access logs, configuration changes, incident records
HIPAA	Six years for required documentation; log retention follows your policy	Access to PHI, authentication events, audit trails
PCI-DSS	One year of audit log history, with the most recent three months immediately available	Cardholder data access, authentication, network events
GDPR	No longer than necessary for the purpose	Data access, consent, deletion requests

```
// Which kind of security event you hold, and which bucket stores it
fetch security.events, from:-7d
| summarize {events = count()}, by:{dt.openpipeline.source, dt.system.bucket}
| sort events desc
```

6. Event-to-Metric Extraction for KPIs

Extract operational KPIs from both business and security events for dashboards and alerting.

Business KPIs

KPI	Metric Source	Extraction Rule
Conversion rate	`purchase.completed` / `product.viewed`	Extract count for each, compute ratio in dashboard
Average order value	`order.total` from `purchase.completed`	Value metric; average at query time
Revenue per hour	`order.total` from `purchase.completed`	Value metric; sum it per hour at query time

Security KPIs

KPI	Metric Source	Extraction Rule
Attack attempts per hour	`DETECTION_FINDING` events	Counter metric; sum per hour at query time
Open vulnerabilities	Vulnerability events	Not a metric-extraction job — metric processors count, sum and histogram; they do not count distinct values. Query `security.events` directly
Failed auth attempts	Authentication events	Extract count where `status == "FAILED"`

```
// Security event trend over 7 days
fetch security.events, from:-7d
| makeTimeseries event_count = count(), by:{event.type}, interval:24h
```

Summary

In this notebook you learned:

- **Business events scope** — Sources, key fields, and discovery queries for transaction data
- **Business event enrichment** — Adding business context, channel attribution, and security context at ingestion
- **Metric extraction from business events** — Conversion funnels, revenue metrics, and operational KPIs
- **Security events scope** — Never drop, never sample, always retain with extended retention
- **Compliance-driven routing** — Dedicated buckets with retention aligned to SOC 2, HIPAA, PCI-DSS requirements
- **Event-to-metric extraction** — KPI dashboards for both business and security monitoring

---

Next Steps

Continue to **OPIPE-06: Cross-Scope Design Patterns** to learn how to correlate data across scopes, cascade processing from spans to events to metrics, and design a production-ready OpenPipeline architecture.

References

- Business event capture (DT docs)
- Get business events from logs and spans (DT docs)
- Business event bucket assignment via OpenPipeline (DT docs)

- Application Security (DT docs)
- Threat Observability concepts (DT docs)
- IAM policy statements (DT docs) — lists `storage:security.events:read`
- Data retention periods (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.